LUMO: Description of lumo-gov-llm-engine.zip

Prepared with Claude · 20 October 2026

1. What this ZIP is

The ZIP contains two layers of the LUMO backend. It supersedes the earlier `lumo-finance-engine.zip`.

- **Layer 1 is the financial engine** (`apps/api/app/finance/`). It is unchanged apart from one line that extends the shared 422 error handler. It is deterministic Python with no LLM, and it computes every rupee figure.
- **Layer 2 is the government-LLM layer** (`apps/api/app/llm/`). It is new. It connects BharatGen and Sarvam models and uses them only for language tasks, never for arithmetic.

Test status: 145 tests, 137 run and pass, 8 skipped. The 8 skipped ones are the HTTP tests (4 in `finance/tests/test_router.py` , 4 in `llm/tests/test_router.py`). They need FastAPI, Pydantic and httpx, which were not available in the sandbox. So `router.py` , `schemas.py` and `main.py` have not been executed.

Size: about 2,365 lines of application code, excluding tests.

2. Architecture: who does what

```
User narrative (Hindi / Hinglish / English)
        |
        ▼
 llm/intake.py ──► government LLM ──► validate ──► (else rule-based demo
extractor)
        |
        ▼
 draft FinancialTwin  +  questions  ──► user reviews / answers
        |
        ▼
 finance/scenarios.py  (deterministic: baseline + 30% shock)
        |
        ▼
 simulation result ──► llm/explain.py ──► government LLM re-words a curated
glossary
                                          using ONLY numbers from the result
```

Job	Done by
Cash projection, reserve breach, shock, goals, all numbers	Deterministic Python
Narrative to structured facts	Government LLM, validated, or rule-based fallback
Plain-language explanation (Hindi, English, Punjabi)	Government LLM, validated, or template fallback

3. Which LLMs and why

Provider	Model	Government link	Access
`bharatgen`	`bharatgenai/Param-1-2.9B-Instruct` (default, configurable)	Government-funded under the Department of Science and Technology (DST) mission NM-ICPS, with IndiaAI Mission compute	Open weights. Self-host with vLLM and set `BHARATGEN_BASE_URL`. No confirmed public hosted API was found.
`sarvam`	`sarvam-30b` (default, configurable)	Selected by the Government of India under the IndiaAI Mission to build the sovereign LLM	Hosted API at `https://api.sarvam.ai/v1` with an API key

Enforcement: `build_gateway_from_env` only accepts `bharatgen` and `sarvam`, plus `demo`, which is ignored. Anything else raises an error that says "Only government-backed providers are allowed".

4. File-by-file description

Financial engine (`app/finance/`)

File	Role
`money.py`	Decimal arithmetic, the single rounding rule, Indian digit grouping (₹12,34,567)
`domain.py`	Plain dataclasses mirroring the FinancialTwin contract
`validation.py`	Structured validation issues and the 422 body

File	Role
`scheduler.py`	Turns once, daily, weekly and monthly flows into dated events
`engine.py`	Day-by-day projection, Stress Radar status, driver ranking
`scenarios.py`	Shock application, baseline and stress runs, comparison, notes
`serialize.py`	Results to contract JSON
`goals.py`	Savings-goal calculator (no endpoint)
`schemas.py`, `router.py`	Pydantic models and `POST /api/finance/simulate`. The error handler now covers `/api/finance`, `/api/explain` and `/api/intake`.
`README.md`, `examples/`	Assumptions, limits, sample requests and responses

LLM layer (`app/llm/`)

File	Role
`errors.py`	Error types: provider not configured, provider call failed, answer rejected, bad input. Bad input produces the same structured 422 shape as the engine.
`text_utils.py`	Converts Devanagari and Gurmukhi digits to ASCII. Finds every number in a text, including `1,23,456`, "2 hazar" and "1.5 lakh". Strips `<think>` blocks. Parses JSON out of messy replies.
`providers.py`	The two HTTP adapters, built on the standard library with no extra dependencies
`gateway.py`	Provider chain, validation, one repair retry, labelled fallback, environment factory
`guards.py`	Number guard, language guard, content block-list

File	Role
`glossary.py`	Four vetted terms and fallback texts in Hindi, English and Punjabi
`explain.py`	LUMO Samjho: context sanitiser, prompt, validator, response builder
`intake.py`	Narrative to draft twin: LLM extraction, validator, rule-based fallback, draft builder
`schemas.py`, `router.py`	Pydantic request models and `POST /api/explain`, `POST /api/intake/compile`, plus `register_llm(app)`
`README.md`, `examples/`	Setup guide, four example request/response files

Other files

- `.env.example`: setting names only, with no secrets.
- `README.md`: top-level pointer and the command to run all tests.
- `app/main.py`: a stub entry point. `/health` reports `mode: "live"` only if a provider is configured, and lists the providers. The integrator replaces it.

5. How each component works

5.1 Provider adapters (`providers.py`)

- Each call is `POST {base_url}/chat/completions` with `model`, `messages`, `temperature` and `max_tokens`. It adds `response_format: json_object` only if `*_JSON_MODE=true`.
- **BharatGen** sends `Authorization: Bearer <key>` only if a key is set, because a self-hosted server may need none.
- **Sarvam** sends `api-subscription-key: <key>`. The key is required, and the header name is configurable.
- **Failures become clean errors** and never include secrets. These are HTTP errors (`HTTP 500`), timeouts, connection failures, an unexpected response shape, a truncated answer (`finish_reason: length`), and an empty answer.

- **Reasoning models** that emit `<think>…</think>` have that block removed. An unclosed block means the answer ran out of tokens, so it counts as empty.
- **Settings:** defaults are timeout 30 seconds, temperature 0, 900 tokens for explanations and 1,500 for intake.

5.2 Gateway (`gateway.py`)

1. Try providers in the configured order. Skip any that are not configured.
2. Parse the JSON reply, tolerating code fences, then run the caller's validator.
3. **If the answer is rejected,** send it back once with the exact reason ("Reply again with ONLY the corrected JSON"). The default is 2 attempts per provider.
4. **If the provider fails over the network,** go straight to the next provider with no retry.
5. **If nothing validates,** run the deterministic fallback. The result is labelled `mode: "demo"` , `provider: "fallback"` , with `warnings` explaining what happened.
6. **If a provider succeeds,** the label is `mode: "live"` and `provider: "<key>:<model>"` .

5.3 Guards (`guards.py`)

- **Number guard.** Every number in the LLM's text must appear in the numbers supplied by the engine. List markers like "1." are ignored. Example: with allowed numbers {500, 2026, 10, 23}, the text "₹500 on 2026-10-23" passes and "₹700 on 2026-10-23" is rejected. Devanagari digits are checked too.
- **Language guard.** Hindi and Punjabi answers need at least 50% of their letters in the right script. English answers may contain at most 10% Devanagari or Gurmukhi letters.
- **Content block-list.** A heuristic list of certainty claims and loan advice in English, Hindi and Punjabi, such as "guarantee", "will definitely", "should take a loan" and "लोन ले". It is a safety net, not a proof.

5.4 LUMO Samjho (`explain.py`)

- **Terms:** only `projected_cash_shortfall` , `minimum_reserve` , `cash_deficit` and `sales_shock` can be explained. The English definition is the source of truth given to the LLM, so the model never decides what a term means.
- **Context is data, not instructions.** Up to 20 keys, each lower_snake_case. Values may only be a boolean, null, a finite number, or a short plain string of 60 characters or fewer using `A-Za-z0-9_-: .` . A sentence like "Ignore previous instructions; …" is rejected.

- **Answer limits:** `explanation` up to 700 characters, `example` up to 450. Both must be non-empty strings.
- **Response:** `language`, `explanation`, `example`, `provider`, and `audio`, plus additive `mode` and `warnings`.
- **Audio:** `not_requested` if not asked for. `unavailable` if requested, because no text-to-speech is wired in. It never fakes audio.
- **Helper:** `explain_context_from_result()` builds the context from a real engine result.

5.5 Narrative intake (`intake.py`)

LLM path. The prompt gives the schema and rules, and the validator checks the answer:

- Every amount must appear in the user's text.
- Every `source_quote` must be a verbatim part of the text.
- Category, direction, frequency and amount basis must be valid values.
- A `business_sales` flow must be an inflow, and a cost must be an outflow.
- A stated date's day-of-month must appear in the text.
- There can be at most 20 flows.

Rule-based demo extractor. It splits the text into clauses, then:

- Classifies each clause by keyword in this priority order: reserve, cash, debt, supplier, operating cost, stock, household, sales. It recognises English, Hinglish and Devanagari words.
- Detects frequency (once, daily, weekly, monthly) and amounts, including "hazar", "lakh" and "k".
- Ignores durations and percentages, so "14 days" and "30%" are not amounts.
- Treats numbers up to 31 followed by "th" or "तारीख", or preceded by "on", as due days and not amounts.

What the builder does:

- Cash and reserve stay `null` if not stated, and a question is created.
- A flow with no known schedule or date is not added to the twin. A `q_frequency_*` or `q_start_date_*` question is asked instead.
- Daily flows start on the as-of date.
- "EMI" or "kist" with no stated frequency is assumed monthly, with a warning and lower confidence.

- A profit or net statement ("my profit is 600") is stored without adding a separate stock cost, with a warning about double counting.
- Questions are in Hindi for `hi-IN`, otherwise English.
- With an `existing_twin`, confirmed values are kept unless restated. A restated value updates the old one, with a warning. Matching is by category plus direction plus frequency, so there are no duplicates.

6. How the values are calculated

6.1 Extraction confidence

Confidence is computed from evidence, never from the model's self-report:

```
base = 0.9 if the LLM answer passed validation, else 0.8 (rule-based)
confidence = base − 0.2 (sales amount basis unclear) − 0.1 (frequency
assumed)
```

Case	Result
LLM, stock cost stated clearly	0.9
LLM, "earn 1500" (basis unclear)	0.7
Rule-based, stock cost	0.8
Rule-based, "earn 1500"	0.6
Rule-based, "EMI 800" assumed monthly	0.7

6.2 Next due date from a day-of-month

- If the day falls on or after the as-of date this month, use this month. Otherwise use next month, clamping to the month's last day.
- With an as-of date of 2026-10-19, "25th" gives **2026-10-25**.
- "5th" has already passed, so it gives **2026-11-05**.

6.3 Scaled amounts

- "2 hazar" = 2 × 1,000 = **2,000**.
- "1.5 lakh" = 1.5 × 100,000 = **150,000**.

- "2.5k" = **2,500**.

6.4 Worked end-to-end example

This is the test `test_narrative_to_simulation`. The input is the Hinglish narrative: "Main sabzi bechta hoon, roz 1500 rupaye bikri hoti hai. Maal pe roz 900 kharcha. Ghar ka kharcha 300 daily. Haath mein 2000 hai. Kam se kam 1000 rakhna hai."

Extraction (demo mode):

Fact	Value	Confidence
Occupation	vegetable vendor	n/a
Cash on hand	₹2,000	n/a
Minimum reserve	₹1,000	n/a
Sales, daily (stated as sales, so basis is total)	₹1,500	0.8
Stock, daily	₹900	0.8
Household, daily	₹300	0.8

Because the sales basis is clear and stock is present, no questions are returned. The sales confidence stays at 0.8 because "bikri" means total sales.

Deterministic simulation (30 days from 19 Oct, with a 30% sales drop for 14 days from 19 Oct):

- **Baseline:** daily net is 1,500 − 900 − 300 = **+₹300**. The lowest balance is 2,000 + 300 = **₹2,300** on 19 Oct. Status is `within_reserve`.
- **Stressed:** sales become 1,500 × 0.7 = ₹1,050. Daily net is 1,050 − 1,200 = **−₹150**. Closing balance on day k is 2,000 − 150k.
  - Day 7 (25 Oct): 2,000 − 1,050 = **₹950**, below the ₹1,000 reserve. This is the first breach.
  - Day 14 (1 Nov): 2,000 − 2,100 = **−₹100**. This is the first cash deficit and the lowest balance.
- **Radar:** shortfall to reserve = 1,000 − (−100) = **₹1,100**. Change in minimum balance = −100 − 2,300 = **−₹2,400**. Status is `cash_deficit`.

6.5 Explanation example

From the finance example's stressed result (shortfall ₹1,200, lowest balance on 2026-11-01), the demo Hindi explanation reads: "इस मामले में सबसे कम नकदी 2026-11-01 को होने का अनुमान है, और वह आपकी सुरक्षित रकम से लगभग ₹1,200 कम हो सकती है।" The ₹1,200 and the date come straight from the engine, so no number is invented. If an LLM answered with ₹700 instead, the number guard would reject it, one repair attempt would follow, and then the template would be used.

The finance engine's own calculations (projection equations, shock window, drivers, goal maths) are exactly as described in the earlier description of `lumo-finance-engine.zip`.

7. Tests (what each file proves)

File	Covers
`test_providers.py`	Headers (Bearer vs `api-subscription-key`), model name, JSON mode, `<think>` stripping, HTTP 500 without leaking the key, empty/null/truncated answers, timeout, connection refused, unconfigured providers
`test_gateway.py`	Live success, JSON in code fences, repair retry containing the rejection reason, failover between providers, demo labelling, environment factory, non-government providers rejected, no keys in `describe()`
`test_guards.py`	Number formats, Devanagari digits, language and block-list checks, fallback templates pass every guard in 3 languages, JSON parsing
`test_explain.py`	Contract shape, fallback labelling, audio statuses, real engine output, valid LLM answer accepted, invented number, wrong language, loan advice, schema violations, context injection rejected

File	Covers
`test_intake.py`	English, Hinglish and Hindi extraction, due-day versus amount, scaled amounts, questions, pending flows, net-income handling, quotes from user text, existing-twin merging, 8 kinds of invalid LLM answer rejected, and the end-to-end narrative-to-simulation test

8. What changed from the previous ZIP

- **Added:** the whole `app/llm/` layer, `.env.example`, a top-level README, and the `/health` stub.
- **Changed:** one line in `finance/router.py` so the structured 422 handler also covers `/api/explain` and `/api/intake`.
- **Unchanged:** all engine logic and its 70 tests.

9. Limits and things to verify

- **No real model has been called.** Tests use a local mock server. A 2.9B model may return bad JSON often, which would show up as fallbacks in the `warnings`.
- **Sarvam's header name and model IDs** came from a third-party guide, not Sarvam's own docs. Check them at docs.sarvam.ai. Sarvam models also reason by default and count reasoning tokens toward `max_tokens`.
- **Licences:** Param-1-2.9B-Instruct is an early checkpoint with an MIT licence per its model card. Domain variants such as FinanceParam are non-commercial.
- **Hindi and Punjabi fallback texts** are drafts and need native-speaker review. Questions in Punjabi fall back to English, and the response marks them `en-IN`.
- **The rule-based extractor** is keyword-based. It does not understand number words like "डेढ़ हज़ार", and it cannot split sentences that have no separators.
- **No audio** is produced.
- **Privacy:** user text is sent to the chosen provider. Sarvam is hosted, so data leaves your infrastructure. Self-hosted BharatGen keeps data on your own servers. Use fictional data only until consent and privacy handling exist.
- **Contract additions** such as `provider`, `mode`, `warnings` and the extra timeline fields need sign-off from the integration owner.

- **Which LLM counts as "government-based"** is the organisers' call. BharatGen is the safer match, because it is government-funded and open-weight.